sorting algorithms

mahmood

February 8, 2023

1 comparison sort

Definition 1.0.1. a **comparison sort** is a type of **sorting algorithm** that only reads the list elements through a single abstract comparison operation (often a "less than or equal to" operator or a three-way comparison) that determines which of two elements should occur first in the final sorted list.

Lemma 1. any comparison-sort based algorithm has a time complexity that is bound by $\Omega(n \log n)$

Example 1.0.1. a vector `A` of size `n` is **defined** to be a "hill" iff for every $i \leq i < j \leq m$ we know $A[i] \leq A[j]$ and for every $m \leq i < j \leq n$ we know $A[i] \geq A[j]$ where m is a constant and i, j are variable indicies
prove that any comparison-based algorithm that turns a vector into a hill has to perform $\Omega(n \log n)$ operations

Answer 0.0.1. we assume in contradiction that there exists a comparison-based algorithm that turns a vector into a hill whose time complexity is less than $\Omega(n \log n)$
consider the following function:

```
SORT(A):
    B ← HILL(A)
    m ← MAX(B)
    C ← REVERSE(B[m, ..., n])
    D ← B[1, ..., m − 1]
    return MERGE(C, D)
```

each line from 3 to 6 runs in $\Theta(n)$ time, except the first which we know by the assumption runs in less than $\Theta(n \log n)$, so the whole function runs in less than $\Theta(n \log n)$ time
this is a comparison-based function that sorts a given array in less than $\Theta(n \log n)$ time, which contradicts with our theorem that no sorting function can have a lower time complexity than $\Theta(n \log n)$
we arrived at a contradiction and so it has to be true that this function is bound from below by $\Theta(n \log n)$, i.e. its time complexity is $\Omega(n \log n)$

2 selection sort

selection sort maintains two subarrays in a given array, one subarray that is sorted and the other unsorted, it sorts a given array by repeatedly finding the minimum element (considering ascending order) from the unsorted part and putting it at the beginning
in every iteration of selection sort, the minimum element (considering ascending order) from the unsorted subarray is picked and moved to the sorted subarray.
time complexity is $\Theta(n^2)$

1 **Algorithm:** selection sort
2 **Function** SelectionSort(A):
3 $B \leftarrow A$
4 **for** $i \leftarrow 1$ **to** $n - 1$ **do**
5 $min \leftarrow$ index of minimal element of $B[i \dots n]$
6 Swap($B[i]$, $B[min]$)
7 **return** B

```
template <typename T>
void selection_sort(T arr[], int len) {
  for (int i = 0; i < len; ++i) {
    // find index of minimum in unsorted subarray
    int min_idx = i;
    for (int j = i+1; j < len; ++j) {
      if (arr[j] < arr[min_idx]) {
        min_idx = j;
      }
    }
```

```
    // add minimum to the sorted subarray
    T tmp = arr[i];
    arr[i] = arr[min_idx];
    arr[min_idx] = tmp;
  }
}
```

example usage:

```
#include <iostream>

int main() {
  int arr[] = {8, 1, 2, 10, 50, 18};
  selection_sort(arr, std::size(arr));
  for (int i = 0; i < std::size(arr); ++i) {
    std::cout << arr[i] << std::endl;
  }
}
```

```
1
2
8
10
18
50
```

3 insertion sort

1. iterate from arr[1] to arr[n]

2. compare the current element to its predecessor

3. if the current element is smaller than its predecessor, drag it backwards until its bigger than its predecessor

time complexity is $O(n^2)$

```
template <typename T>
void insertion_sort(T arr[], int len) {
  for (int i = 1; i < len; ++i) {
    int idx = i;
    while (idx > 0 && arr[idx] < arr[idx-1]) {
      T tmp = arr[idx];
      arr[idx] = arr[idx-1];
      arr[idx-1] = tmp;
      idx--;
    }
  }
}
```

4 AVL sort

AVL sort consists of inserting the given array into an AVL tree element by element and then converting the tree into an array using inorder traversal

Lemma 2. the time complexity of this algorithm is $\Theta(n \log n)$

5 heap sort

heap sort consists of transforming the given array into a heap array `A`, initiating `heap-size` to `size(A)` then applying the following steps recursively:

1. replace the root of the heap `A[0]` with `A[heap-size - 1]`

2. bubble down the newly replaced root (`A[0]`)

3. decrement `heap-size`, return to step 1 if `heap-size > 1`

Lemma 3. the time complexity of this algorithm is $\Theta(n \log n)$

note

on the third step when decrementing `heap-size` we arent deleting the last element, we are simply decreasing the size of the part of the list that we're operating on because the part at the end contains elements that are sorted

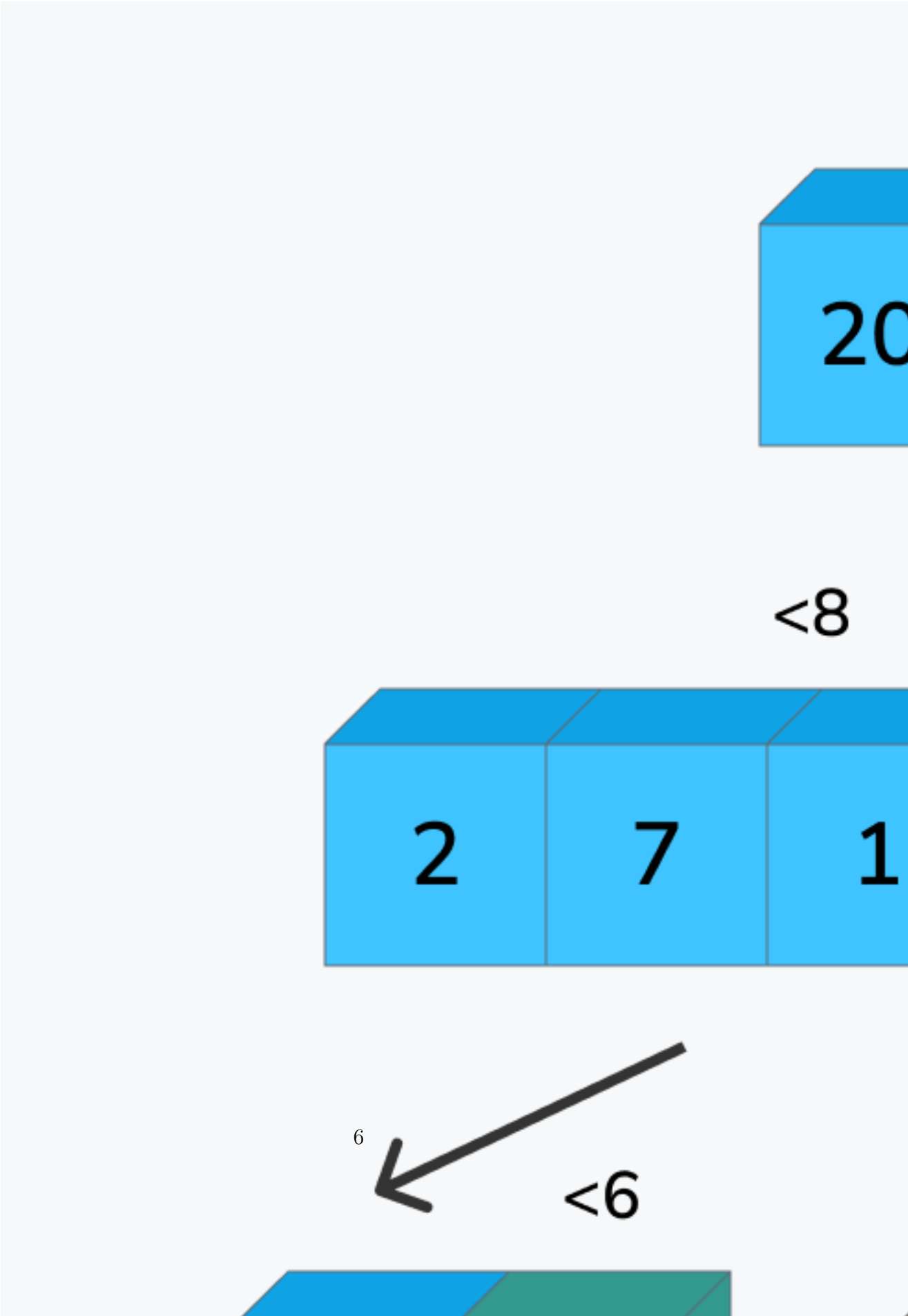
6 QuickSort

QuickSort is a Divide-and-conquer algorithm, it picks an element as **pivot** and **partitions** the given array around the pivot, there are different versions of quicksort that pick pivots in different ways:

1. first element as pivot (implemented here)
2. last element as pivot
3. a random element as pivot
4. median as pivot

the first subarray we use as a "partition" is the array itself, where given a pivot, put the pivot at its correct position in the sorted array by putting all the smaller elements before it, and all the greater elements after it. then divide the partition in 2 partitions, where the first partition is where all the smaller elements were put, and the second partition is where all the greater elements were put, and apply the same process for these partitions recursively until we reach partitions that contain 1 element only
this algorithm runs in $O(n \log n)$ time
image taken from `https://workat.tech/problem-solving/tutorial/sorting-algorithms-q`

```
template <typename T>
void swap(T arr[], int i, int j) {
  T tmp = arr[i];
  arr[i] = arr[j];
```


20
<8
2
7
1
6
<6

```
  arr[j] = tmp;
}

template <typename T>
int partition(T arr[], int low, int high) {
  int pivot = low++;
  while (low <= high) {
    if (arr[low] <= arr[pivot]) low++;
    else if (arr[high] >= arr[pivot]) high--;
    else swap(arr, low, high);
  }
  swap(arr, high, pivot);
  return high;
}

template <typename T>
void quick_sort(T arr[], int low, int high) {
  if (low <= high) {
    int pivot = partition(arr, low, high);
    quick_sort(arr, low, pivot-1);
    quick_sort(arr, pivot+1, high);
  }
}
```

7 merge sort

yet another divide-and-conquer algorithm
given an array, this algorithm keeps splitting it until it reaches a subarray that cant be divided which happens when a subarray contains only 1 or 0 elements, each of these subarrays are sorted individually and then combined, recursively, to eventually make a larger sorted array
this algorithm runs in $\Theta(n \log n)$ time
`./merge_sort.webp`
image taken from `https://www.programiz.com/dsa/merge-sort`, will keep this here until i make my own

```
#include <iostream>
template <typename T>
void merge(T arr[], int l, int m, int r) {
```

```
  int n1 = m - l + 1;
  int n2 = r - m;
  T L[n1];
  T R[n2];

  for (int i = 0; i < n1; ++i)
    L[i] = arr[l + i];
  for (int j = 0; j < n2; ++j)
    R[j] = arr[m + 1 + j];

  int i = 0, j = 0;
  int k = l;
  while (i < n1 && j < n2) {
    if (L[i] <= R[j])
      arr[k++] = L[i++];
    else
      arr[k++] = R[j++];
  }

  while (i < n1)
    arr[k++] = L[i++];
  while (j < n2)
    arr[k++] = R[j++];
}

template <typename T>
void sort(T arr[], int l, int r) {
  if (l < r) {
    int m = l+(r-l)/2;
    sort(arr, l, m);
    sort(arr, m + 1, r);
    merge(arr, l, m, r);
  }
}
```

7.1 merge

consider the `merge` algorithm that merges two partitions, i.e. takes in two sorted arrays A,B, and returns a new sorted array C that contains all elements from A and B

```
MERGE(A, B):
    A[n+1] ← B[m+1] ← ∞
            // to ensure we dont go over A and B's boundaries
    ap ← bp ← cp ← 1  // pointers for each of the arrays A,B,C
    while cp < m+n+1 do
        if A[ap] ≤ B[bp] then
            C[cp] ← A[ap]
            ap++
        else
            C[cp] ← B[bp]
            bp++
        cp++
    return C
```

8 bubble sort

this algorithm repeatedly steps through the input list element by element, comparing the current element with the one after it, swapping their values if needed. these passes through the list are repeated until no swaps had to be performed during a pass, meaning that the list has become fully sorted. the algorithm, which is a comparison sort, is named for the way the larger elements "bubble" up to the top of the list.

```
Algorithm: bubble sort
Function BubbleSort(A):
    for Lim ← n to 2 do
        for j ← 1 to Lim − 1 do
            if A[j] > A[j+1] then
                Swap(A[j], A[j+1])
```